Propuesta de Proyecto: Motor de Procesamiento Distribuido sobre Kubernetes para Análisis de Riesgo en Contratación Pública

Integrantes

- Salomon Avila
- Andrés Felipe Caro
- Eliana Pardo
- Gabriela Bonilla
- Samuel Montaña
- Lorenzo Ramirez Calderon

Objetivo general

Diseñar e implementar un sistema distribuido nativo bajo una arquitectura Master-Worker con 1 nodo maestro y 3 nodos de trabajo, basado en el paradigma MapReduce, orquestado sobre un clúster Kubernetes. El sistema procesará los conjuntos de datos abiertos de SECOP II de la Agencia Nacional de Contratación Pública para construir un indicador conjunto de riesgo en la contratación pública colombiana que combine nivel de competencia real del proceso de selección con concentración de proveedores y desviación de precio por categoría. El proyecto evaluará el rendimiento algorítmico, la tolerancia a fallos y la escalabilidad del procesamiento paralelo, incluyendo el patrón de join distribuido entre dos conjuntos de datos masivos, frente a enfoques secuenciales tradicionales.

Fuentes de datos

La información proviene de dos conjuntos de datos del portal datos.gov.co, suministrados por Colombia Compra Eficiente:

Procesos de Contratación contiene 9,14 millones de filas y 59 columnas, con actualización diaria. Registra el embudo completo de competencia por proceso: número de proveedores invitados, proveedores que manifestaron interés, y proveedores únicos que presentaron respuesta.

Contratos Electrónicos contiene 95 columnas con información del contrato ya adjudicado, incluyendo identificación del proveedor ganador por NIT, entidad contratante por NIT, valor total del contrato, categoría UNSPSC y modalidad de contratación.

La llave de unión entre ambos conjuntos es id_del_proceso en Procesos equivalente a proceso_de_compra en Contratos.

El volumen total en disco de ambos conjuntos, validado por muestreo previo, es del orden de 20 a 40 GB en formato crudo sin comprimir, muy por debajo de la capacidad del clúster de 300 GB de almacenamiento y 192 GB de RAM agregados. Por tanto, el volumen de datos no es la restricción de diseño; el reto central está en la lógica de shuffle y de join distribuido.

Objetivos específicos

1. **Desplegar la infraestructura de orquestación** configurando un clúster Kubernetes con 1 nodo master y 3 nodos worker que gestione el ciclo de vida de los contenedores del sistema distribuido. Se garantizará descubrimiento de servicios estable entre el plano de control

y el plano de datos mediante un StatefulSet con Service headless para los workers y un Deployment para el master.

2. **Implementar el motor distribuido** desarrollando desde cero, en un lenguaje de alta concurrencia como Go, el orquestador en el master y los ejecutores en los workers del motor MapReduce. El motor incluirá partición de cada conjunto de datos en chunks, distribución equitativa entre workers, protocolo de comunicación Master-Worker mediante RPC o HTTP para asignación de tareas, heartbeats y recolección de resultados, fase de shuffle con agrupación de resultados intermedios por clave, y mecanismo de tolerancia a fallos a nivel de aplicación mediante detección de workers caídos por timeout de heartbeat y reasignación automática de tareas pendientes.

   Kubernetes se limitará a garantizar que los procesos estén vivos y ubicados en nodos distintos mediante podAntiAffinity. La lógica de particionamiento, scheduling de tareas y tolerancia a fallos a nivel de aplicación será responsabilidad exclusiva del motor propio, no de primitivas nativas de Kubernetes como Job con completions indexados.

3. **Implementar la lógica de análisis de riesgo** paralelizando el cálculo del indicador conjunto en dos etapas encadenadas sobre el motor genérico MapReduce.

   En la Etapa 1, dos jobs se ejecutan en paralelo sin dependencia entre sí. El Job A procesa Procesos de Contratación: la fase Map emite tuplas con id_del_proceso, proveedores_invitados y proveedores_unicos_con_respuestas; el Reduce calcula el índice de competencia real definido como la razón entre proveedores únicos con respuesta y proveedores invitados por proceso. El Job B procesa Contratos Electrónicos: la fase Map emite tuplas con id_proceso, nit_proveedor, nit_entidad, valor_del_contrato y categoría UNSPSC; el Reduce calcula dos métricas por contrato: la desviación de precio respecto a la mediana de su categoría, y la concentración proveedor-entidad como porcentaje de contratos de un proveedor con una misma entidad.

   En la Etapa 2, se ejecuta un join distribuido mediante reduce-side join sobre los resultados agregados de la Etapa 1, no sobre los datos crudos. La fase Map etiqueta cada registro intermedio según su origen y emite tuplas con id_proceso y datos etiquetados. El shuffle agrupa por id_proceso. El Reduce combina datos de ambos orígenes para cada proceso, produciendo el indicador conjunto de riesgo. Se maneja explícitamente el caso de procesos sin contrato asociado como left outer join, conservando dichos registros como hallazgos propios.

4. **Producir un reporte consolidado de riesgo** que entregue un ranking de procesos, proveedores y entidades según su concurrencia en tres dimensiones: índice de competencia real bajo, desviación de precio relativa hacia arriba, y relación proveedor-entidad recurrente en el tiempo. El reporte será exportable en formato CSV para consumo por herramientas de visualización y análisis posterior. Este reporte constituirá una señal reproducible sobre cualquier período del histórico de SECOP II, replicable de forma abierta y comparable a indicadores que hoy calculan de forma manual organizaciones de veeduría ciudadana y periodismo de datos.

5. **Medir y analizar métricas de rendimiento** cuantificando la eficiencia del clúster mediante speedup definido como T_1 dividido entre T_n y eficiencia como speedup dividido entre n, variando el número de workers activos entre 1, 2 y 3. Se medirán estas métricas por separado para la Etapa 1 (agregación) y la Etapa 2 (join), dado que tienen perfiles de carga distintos. Además, se medirá throughput en registros o MB procesados por unidad de tiempo, volumen de tráfico de red generado específicamente por el shuffle del join como

indicador del costo de comunicación del patrón reduce-side join frente a la agregación simple de la Etapa 1. Se medirá tiempo de recuperación ante fallos, distinguiendo explícitamente entre recuperación de la tarea medida por el master al reasignar el chunk de un worker caído, y recuperación del proceso medida por el tiempo que tarda Kubernetes en reprogramar un pod eliminado según sus livenessProbe y readinessProbe.

Justificación y contexto

La contratación pública en Colombia se publica de forma abierta a través de SECOP II, pero de manera fragmentada: el detalle de la competencia de cada proceso de selección y el detalle del contrato finalmente adjudicado viven en conjuntos de datos separados, sin que exista un indicador público consolidado que los cruce. Calcular una señal conjunta de riesgo que combine baja competencia real, sobreprecio relativo y concentración de proveedores, a la escala de los más de 9 millones de procesos publicados, es un problema computacional intensivo. El reto no es solo el volumen de cada fuente por separado, sino especialmente el costo de unir ambas fuentes de forma distribuida, siendo este el caso de uso canónico del patrón reduce-side join en el paradigma MapReduce.

Este proyecto aborda dicho problema construyendo un motor de procesamiento distribuido desde cero, desplegado sobre un clúster Kubernetes real. La separación de responsabilidades es deliberada: Kubernetes resuelve el problema de virtualización y orquestación de procesos mediante ciclo de vida de contenedores, red, ubicación en nodos y recuperación de procesos caídos. El motor propio resuelve el problema de distribución de datos y cómputo mediante partición, comunicación, shuffle, join distribuido y tolerancia a fallos a nivel de tarea. Ambas capas son complementarias y se estudian por separado en el análisis de resultados, evitando así duplicar con código propio funcionalidad que Kubernetes ya provee de forma nativa, y evitando también delegar en Kubernetes la lógica que es objeto de aprendizaje del curso.

A diferencia de utilizar frameworks de procesamiento distribuido preexistentes como Spark o Hadoop, el desarrollo de un motor propio sobre infraestructura orquestada permite control absoluto sobre el uso de hilos del procesador, gestión de memoria e tráfico de red del plano de datos, ofreciendo un entorno controlado para extraer métricas puras sobre cómo se comportan sistemas concurrentes ante cargas de trabajo masivas, incluyendo el costo específico de un join distribuido frente a una agregación simple, sin las capas de abstracción que dichos frameworks introducen.

Adicionalmente, el resultado del proyecto trasciende la demostración técnica: el indicador conjunto de riesgo producido es un insumo de valor público real, replicable sobre cualquier período del histórico de SECOP II, en una línea de análisis hoy reservada a organizaciones de veeduría ciudadana y periodismo de datos con recursos dedicados a ese cruce manual. La herramienta generada abre la posibilidad de monitoreo continuo y análisis comparativo de patrones de riesgo en la contratación pública colombiana a nivel nacional, regional y sectorial.